

Design as the Objective: Making Architectural Quality an Explicit Target for Coding Agents

Eliezer Avihail
github.com/eliezeravihail/aims

September 2026

Abstract

Coding agents are given one goal: code that works. We argue that a language model pursues the goal it is given rather than the instructions around it, so structural quality will neither be created nor preserved unless it becomes part of the goal, stated as checkable criteria and applied before code is written. AIMS implements this through a scored form built from established design principles, applied both to new designs and to feature additions. We could not show that it helps. When adding features to existing code, agents with and without the method were equally correct, and a judged advantage over a sequence of additions was not confirmed by an automatic measure. For new designs, two comparisons with a spec-driven tool pointed in opposite directions, and the same unchanged designs received grades from 5.7 to 9.8 depending on the judge. Every judge recognized which method produced each design, and the one measure that does not depend on our principles favoured the other tool. There is no accepted measure of design quality, and the automatic metrics we tried rank designs by size rather than structure. Until a measurement exists that gives the same answer twice, the effect of making design the objective cannot be established.

1 Introduction

A coding agent given a clear requirement usually writes code that works. Given a sequence of requirements over many sessions, the code tends to degrade: the same concept is modelled in two places, an interface built for one purpose is stretched to serve another, a rule that had one owner gets a second. Each change passes its tests, and the cost appears only when the next change is harder than it should be.

Our claim is simple. *A model optimizes for the goal it is given.* Agents are given correctness as their goal, so structure is shaped only indirectly, by the ordinary code the model learned from. If good structure is wanted, it has to be the goal: written down as specific, checkable items, and checked when the design is made rather than after the code exists.

2 Why the Goal Must Be Explicit

The model follows the goal, not the wording. A language model produces text one token at a time, choosing what is likely given the context. It has no separate objective for the finished program. Preference training [13] makes it produce answers that *look* like they satisfy the request, which can diverge from what was actually wanted [16]; optimizers in general satisfy the literal specification

they are scored on [10]. For coding agents that specification is correctness: benchmarks such as HumanEval [2] and SWE-bench [9] check whether tests pass, and agent loops [20, 19] stop when the task works.

An instruction such as “write clean, well-structured code” does not change this. It gives the model nothing it can fail. A goal becomes real only when it is a list of items, each of which can be answered yes or no for a given design.

The check must happen before the code. Once the model has written an interface, everything after it is conditioned on that interface. Asking the model afterwards whether the interface is right tends to produce a defence of it rather than a revision. This matches the finding that models correcting themselves without an external signal do not improve and often get worse [8]. It is also cheaper: fixing a design costs a paragraph, fixing code means rewriting it. So the structural check belongs inside the design step, and it needs an external reference — the written list — rather than the model’s own opinion.

This second point has not yet been tested directly; the test is described in Section 11.

Why Aims leans toward more structure. Most code a model has learned from is ordinary working code: long procedures, raw strings and numbers standing in for domain concepts, rules enforced in several places. With correctness as the only goal, nothing pushes away from this. Too much structure costs a layer nobody uses; too little spreads, because every later change has to work around the missing boundary. AIMS therefore treats missing structure as the more serious error when in doubt.

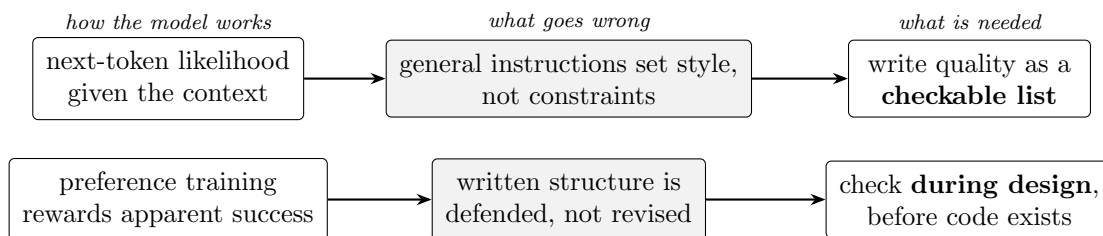


Figure 1: The argument of this section.

3 What Aims Does

AIMS is a plugin for terminal coding agents; it adds skills and commands and does not replace the agent’s own mechanisms. It serves two tasks, and both rely on one measurement.

Designing new code. The aim is a design with good structure from the start: concepts represented as what they are, each rule with one owner, boundaries between modules that later changes will not have to break.

Adding a feature to existing code. The aim is to add the feature so that it fits the existing design and style, and to leave the rest of the behaviour unchanged. This task has a well-documented risk of its own. Lehman’s law of increasing complexity states that a program that keeps changing becomes more complex unless work is done to prevent it [31]. Parnas described how changes made by people who do not understand the original design concept gradually destroy its structure [32].

Eick et al. measured this over fifteen years of change history of a large telephone-switching system and found that changes grew wider, modularity broke down, and changes became more likely to introduce faults [33]; van Gurp and Bosch describe the same process at the level of design as design erosion [34]. It is also known informally as *software entropy*. An agent adding a feature is in exactly the position Parnas describes: it did not make the original design, may not understand it, and is judged only on whether the new feature works.

Measurement. Both tasks use the same scored form. The form is what the builder is asked to satisfy, what the review checks, and what we use to compare designs. It is described in Sections 4–5.

AIMS also includes a tool for documenting design decisions, which is outside the scope of this paper.

4 Where the Rules Come From

The rules are drawn from the established software engineering literature: readability at the level of functions and names from *Clean Code* [22] and *Code Complete* [23]; avoiding duplication, orthogonality and the Law of Demeter from *The Pragmatic Programmer* [24]; the catalogue of code smells from Fowler [5]; design patterns from Gamma et al. [25]; the SOLID principles, component principles and layering from Martin [26]; value objects and domain modelling from Evans [27]; contracts from Meyer [28]; the separation of correctness, performance and security into distinct qualities from ISO/IEC 25010 [29]; and secure coding from OWASP [30]. For adding features, the rules draw on Beck’s advice to first make room for a change and then make it [1], on Feathers’ practice of understanding existing behaviour before editing it [4], and on Fowler [5].

From these sources we kept only what can be checked as yes or no on a given design by a reader; nothing is computed from a line count or a tool. The result is two documents: **design principles** for new code (15 chapters: foundations, correctness and contracts, functions, naming, types and domain modelling, encapsulation, coupling and cohesion, change and extensibility, code smells, patterns, architecture, state and concurrency, testability, performance, security), and **add-feature principles** for changes to existing code (12 chapters), which build on the first rather than replace it.

The documents were then refined by experiment: when a pilot showed a failure the list did not catch, an item was added or sharpened. Three examples:

- **Check every combination, not just the listed cases.** For each rule and each way the product can change, check what their combination implies. The failure this prevents is a required result that no listed test case exercises.
- **Model a concept as what it is.** Do not model something as a degenerate version of a neighbouring type — a tax as a zero-value discount, a blocked slot as a fake booking, a filter as a very low score. Such designs pass every test and break at the next change. This check runs on the design, before code.
- **A fixed rule for “add a boundary” versus “remove what is unused”.** When the two conflict, decide by the expected changes: a boundary that no expected change would use counts as a minor fault; a missing boundary that an expected change would force open counts as a serious one. This takes the decision away from the reviewer’s taste.

Two rules apply to adding features in particular. Success is measured as before-versus-after on the code being changed, not against an ideal. And the existing style takes precedence over the principles, because introducing a foreign style into existing code is itself a regression.

5 From Rules to a Scored Form

Each chapter of the documents is a row in the form, and each item in a chapter is a pass/fail check. Every item also has a class that says how serious its failure is: *preconditions*, whose violation makes the code wrong (functional correctness, one owner per rule, leaks across module boundaries, and concurrency and security where they apply); *conditional* items, checked only when the product states the need; and *quality* items. Given a design and the specification it was made for, the reviewer fills in the form as follows. We use a booking service as a running example.

1. List what the design must satisfy, from the specification. Before looking at the design, the reviewer reads the specification and writes three short lists:

- **Rules** the product must always obey (R). *Example: a room cannot have two bookings at overlapping times.*
- **Directions of change** the product is likely to take, plus one plausible change nobody has stated (X). *Example: bookable resources other than rooms; cleaning time between bookings.*
- **Acceptance cases** the product must pass (C). *Example: a booking from 10:00 to 11:00 succeeds right after one from 9:00 to 10:00.*

These lists come from the specification, not from the design, and they are written once per product. Every design for that product is then checked against the same lists, so a reviewer cannot choose, after seeing a design, which requirements to hold it to.

2. Go through the form item by item. For each item the reviewer marks *pass*, *fail*, or *not applicable*; a typical review has 15–25 applicable items. A failure must point to something specific: one of the R, X or C entries, or a named part of the design. *Example: fails “one owner per rule” — the no-overlap rule (R) is checked both in the booking service and in the calendar import.* A failure without such a pointer is not counted, and one defect is counted under one item only.

3. Score each chapter. A chapter’s score is $\text{round}(10 \times \text{passed}/\text{applicable})$, and it cannot exceed the maximum set by its most serious failure (Table 1).

4. Report. The chapters are combined into a weighted average. It is reported together with the worst chapter, the number of serious failures, and a gate: if any item failed at the most serious level (S4), the design is marked BLOCKED — not shippable until that item is fixed. The grade is never capped and never reported alone.

The same filled form is used in two ways. The designer receives only the list of failures, most serious first, each with its pointer and what to fix — without a score, so that it fixes problems rather than optimizes a number. For comparing designs, the full scored form is used. Because every design of a product is checked against the same lists, grades for the same product can be compared; grades for different products cannot.

6 How the System Works with the Form

New design. A **Guide** sets one design objective for the round and hands the work to **Workers**. For the first round of a product, three Workers design independently, each focused on one aspect — clean code, encapsulation, genericity — and a **merge agent** combines the strongest parts of each;

Table 1: Severity. The worst failure in a chapter sets its maximum score and its weight.

Worst failure in the chapter	Maximum	Weight
none	10	$\times 1$
cosmetic (S1)	8	$\times 1$
moderate (S2)	7	$\times 2$
structural (S3)	5	$\times 4$
correctness / precondition (S4)	2	$\times 8$

No overall cap: the grade is the weighted average; any S4 marks the design BLOCKED.

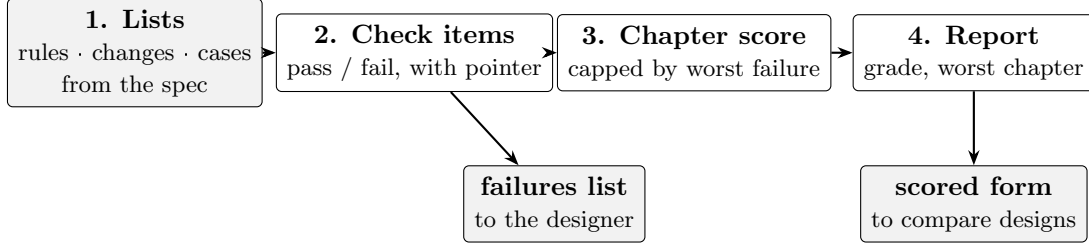


Figure 2: Filling in the form. The lists in step 1 come from the specification, not from the design under review.

later rounds use a single designer. The design is then reviewed with the form before any code exists. The designer receives the list of failures, revises the design once, and only then builds (Figure 3).

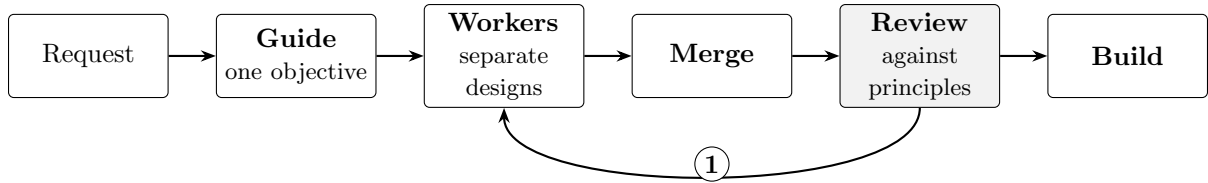


Figure 3: The AIMS loop. The design is reviewed before any code is written; the curved arrow is the single revision round.

Adding a feature. The agent first establishes what the existing code does and where the new feature belongs. If the existing design has no room for it, it first makes room without changing behaviour, and then adds the feature. The form is filled twice, before and after the change, on the code being changed; the change passes if nothing got worse and the feature fits the existing design and style.

7 Why a Form and Not Automatic Metrics

An automatic metric would avoid the risk that a judge favours the method, and no external benchmark or independent judge for design quality exists. We therefore tested the automatic metrics that are available.

Standard metrics. Four automatic measures are commonly used to judge code quality, and tools compute them directly from source code (we used `radon`, a standard tool for Python):

- **Lines of code** — the number of non-blank, non-comment lines. The assumption is that less code is easier to maintain.
- **Cyclomatic complexity** [12] — the number of separate paths through a function, roughly one plus the number of decision points (`if`, loops, and so on). A function with many paths is harder to understand and to test.
- **Halstead measures** [6] — counts of the operators and operands in the code, combined into estimates of its “volume” and of the effort needed to understand it.
- **Maintainability Index** [3] — a single score from 0 to 100 that combines the three measures above into one formula; higher is meant to be more maintainable.

All four measure size and branching. None of them looks at whether the code represents the problem well.

We had one pair of working implementations of the same task that differed in a single design choice: one returned a named type, the other a plain tuple, which a blind judge marked as the weaker design. Table 2 compares the four measures with that judgement.

Table 2: Standard metrics against a blind review, two implementations of one task.

	named type	plain tuple
Blind review (0–4)	4.0	3.5
Maintainability Index	80.01	80.50
Cyclomatic complexity (avg / max)	2.07 / 4	2.07 / 4
Halstead (volume / effort / difficulty)	36 / 48 / 1.33	36 / 48 / 1.33
Lines of code	36	27

Two metrics saw no difference, and two preferred the weaker design, because a named type adds lines without adding branches. This is a single pair, but the reason is general: these metrics count size and branching, and good structure often adds size.

A coupling metric: the idea. Coupling is the degree to which one part of the code depends on details of another, so that changing one forces a change in the other. Page-Jones [14] proposed ranking such dependencies by strength. An example: a function returns a price and a tax. If it returns them as a pair and the caller takes “the second item” to get the tax, the caller depends on the *order* of the items; if someone swaps them, the caller breaks without warning. If it returns an object with a field called `tax`, the caller depends only on the *name*, which is a weaker and safer dependency. Depending on order is stronger than depending on a name, and logic written twice is stronger than both, because every change to it must be made in two places.

How we built it. We wrote a tool that counts the strong dependencies in code: values read by position, “magic” numbers and strings inside comparisons, repeated code, and one rule checked in several functions. A lower count is better. Unlike the standard metrics, this one does not penalize adding a named type, because such a type replaces a dependency on order with a dependency on a name. It ranked the pair in Table 2 correctly: 0 for the named type, 3 for the tuple.

What it showed. Across the seven code comparisons we have, it gave AIMS two wins, three ties and one loss. On the sequence experiment (experiment 4 in Section 8.3) it disagreed with the judge: by this measure the code in both arms got worse over the three changes, the AIMS code slightly less so (a count of 19 against 22 after the third change), whereas the judge described an improvement.

Why we did not adopt it as a quality measure. The one loss shows the limit of any measure that reads only code. In that case the code had to pick the few items with the largest values from a list. Instead of selecting them, it sorted the whole list — and to sort it, it built (value, position) pairs and read them by position. The measure correctly flagged the reading by position. But that was not the real problem: the real problem was the choice of operation. Had the code been written as a selection rather than a sort, the pairs would not exist at all. The measure saw the symptom, not the cause. Seeing the cause requires knowing what the code is *supposed* to do, and that is written in the specification, not in the code. So any measure that reads only the code will miss faults of this kind.

Conclusion. We found no automatic metric that can judge design quality. The coupling metric is useful for pointing at code worth reading, and it can disagree with the judge, which is valuable; it should appear in the findings list, not in the score. Design quality has to be judged against the specification, with the safeguards described in Section 8.1.

8 Experiments

8.1 Protocol

Question. When a product grows through requirements that were not known in advance, does the AIMS design end up better structured than one produced by an equally capable agent without the method, even though that agent is free to restructure the code at every step?

Setup. All arms start from the same code. Requirements arrive in stages; a stage card must not hint that further stages exist. Hidden tests check correctness and are derived only from requirements already given. Questions from the agents are answered by an operator acting as an ordinary product owner, from a fixed hidden specification, with every question and answer logged.

Judging. Designs are anonymized and relabelled. Judges are separate agents that built nothing, and the mapping from labels to arms is sealed until judging is done. Every significant claim must point to a specific line or a reproducing input. Each measure is reported separately and never combined into one number.

Measures. (1) Hidden correctness tests: pass or fail. (2) Components reopened or discarded when a new requirement arrives, counted from the diff (lower is better). (3) The graded profile from the review instrument. Measures (1) and (2) do not depend on our principles; (3) does.

All runs used the current method, one run per arm per product, with all arms and judges on the same model.

8.2 Study 1: new designs, compared with OpenSpec

Setup. Two arms — AIMS and OpenSpec — design three services: feed ranking, booking availability, and entitlements. Design only, no code. **D1** scores the first design; **D2** scores the design

after a new, unannounced requirement. The OpenSpec arm used the installed tool (version 1.13.2), each stage in a separate session that could not see the AIMS repository. An earlier run in which the OpenSpec arm only imitated the tool’s format is not reported.

The comparison was run twice, with the same OpenSpec designs both times. In the **first round**, the AIMS designs came from an earlier run of the method, and one judge scored each product. In the **second round**, AIMS was run again with its current version, on the same model as the OpenSpec arm and in stricter isolation, and two judges of opposite leanings — one favouring clear ownership, one favouring simplicity — scored each product.

Table 3: Study 1, both rounds. Round 2 has two judges per product, so six readings.

	Round 1 (3 readings)	Round 2 (6 readings)
D1 winner by the form	no clear advantage in all 3	AIMS 4, no clear advantage 2
D2 winner by the form	OpenSpec 3	AIMS 5, OpenSpec 1
Fewer components reopened or discarded	AIMS 1, OpenSpec 2	OpenSpec 5, tie 1
Products with a wrong-output failure found	AIMS 3, OpenSpec 0	AIMS 1, OpenSpec 3

The rounds disagree. By the form, OpenSpec did better in the first round and AIMS in the second. Table 4 shows why this cannot be read as a result: the OpenSpec designs were identical in both rounds, yet their grades moved by up to 3.3 points between judges — more than most differences between the arms. Correctness findings did not repeat either: the second round found three wrong-output failures, each with a reproducing input, in OpenSpec designs that the first round had passed.

Table 4: Grades given to the *same, unchanged* OpenSpec designs.

	Round 1	Round 2, ownership judge	Round 2, simplicity judge
feed ranking, D1	8.69	5.92 (blocked)	6.09 (blocked)
booking availability, D1	9.79	6.82 (blocked)	9.47
entitlements, D1	9.27	8.74	9.33
feed ranking, D2	8.69	5.80 (blocked)	5.78 (blocked)
booking availability, D2	9.71	6.52 (blocked)	9.07
entitlements, D2	9.00	5.71 (blocked)	6.36 (blocked)

The judges knew which arm was which. Before scoring, each judge was asked to guess which design came from which method. All nine judges guessed correctly, each at 97% confidence, from OpenSpec’s file layout and requirement wording and from the AIMS designs’ references to its principles.

The measure that does not depend on our principles disagrees with the form. In the second round the form favoured AIMS, but the count of components reopened or discarded favoured OpenSpec in five of six readings.

Cost. In the second round AIMS used 8–13 million tokens, 26–49 minutes and \$12–21 per product, including its Workers. OpenSpec used 1.6–2.2 million tokens and 6–7 minutes.

8.3 Study 2: adding features to existing code

Setup. Five experiments in which a feature was added to existing code, comparing AIMS with an agent without the method, using the add-feature principles and a before-versus-after review of the changed code. One used a real open-source module.

Table 5: Study 2. “Equal” means both arms added the feature correctly and left the rest of the behaviour intact.

#	Code	Feature added	Result
1	synthetic pricing	per-line tax and allocation	equal
2	three synthetic modules	new operator / cancel event / allocation	equal; AIMS kept each rule in one place
3	real open-source cache	add expiry across three entry paths	equal (23 tests, 14 hidden cases); AIMS 31 lines vs. 53
4	synthetic pricing	three changes in sequence	equal at each step; AIMS design improved over the sequence, the other did not
5	four-module ledger	multi-currency	equal; similar designs

Results. The agent without the method added the feature correctly every time. The method did not catch any bug the other agent missed. For a feature that fits in one session, AIMS is not needed for correctness.

Where it helped was over a sequence of additions. In experiment 4, reviewed before and after each of three successive features, the AIMS code got cleaner at each step, while the other arm’s code grew denser — an unnamed twenty-line block, a five-element tuple read by index. A single change does not show this. Also, when the existing code was written in a plain style, AIMS kept that style instead of imposing its own.

Limitations of this study. The improvement in experiment 4 comes from a judge’s review; the coupling measure in Section 7 shows both arms getting worse, AIMS less so. All codebases were small enough to fit in one session.

9 What We Could Not Show, and Why

We set out to show that making design the objective produces better code. We could not show it, and the reason is measurement, not a result in either direction.

Judges do not agree. The same designs received grades from 5.7 to 9.8 depending on the judge (Table 4), and correctness failures found by one judge were missed by another. A difference between methods smaller than this spread cannot be told apart from the choice of judge, and most differences we measured were smaller.

Blinding is not possible with current formats. Each method leaves its own marks in its output, and every judge recognized them. A judge who knows which design came from which method is not blind, whatever the protocol says.

The form shares its source with the method. The form is built from the same principles AIMS designs toward, so it is likely to favour AIMS. The one measure that does not depend on those principles — components reopened when a new requirement arrives — favoured the other tool in the second round.

No outside measure exists. There is no accepted benchmark or independent judge for design quality, and the automatic metrics we tested measure size and branching, which good structure often increases (Section 7).

What would be needed. Three things, none of which we have yet: a measurement of how much independent judges agree, run on the same designs; a way to hide which method produced a design, for example by rewriting all designs into one neutral format before judging; and an outcome that does not depend on any rubric, such as the cost of a long sequence of later changes, measured over more changes than a single session can hold.

10 Related Work

Agent loops. ReAct [20], Self-Refine [11], Reflexion [17] and SWE-agent [19] all stop on task success. Their self-critique uses the same goal as the task and runs after the output exists.

Multi-role systems. AutoGen [18], ChatDev [15] and MetaGPT [7] split work between roles; MetaGPT’s roles exchange structured documents, which is closest to our planner/builder split. There the split speeds up task completion. In AIMS its purpose is that the planner’s output is a design checked against a written standard before building starts.

Spec-driven development. OpenSpec, GitHub Spec Kit and Kiro’s spec mode add a requirements $\rightarrow$ design $\rightarrow$ tasks stage before coding. This is the closest alternative and our baseline. It has a design step, but no written definition of good structure to check that step against.

Practice for changing existing code. The part of AIMS for adding features to existing code draws on standard practice: first make room for the change without altering behaviour, then make the change [1]; understand existing behaviour before editing it [4]; and name any code smell being removed [5].

Metrics and LLM judges. Cyclomatic complexity [12], Halstead volume [6] and the Maintainability Index [3] are the usual automatic measures. LLM-as-judge [21] is the usual alternative. Section 7 shows why the first does not work here, and Section 11 discusses the risks of the second.

11 Limitations

Same model throughout. All judges ran on the same model family as the arms. A judge from another model family has not yet been used.

The two requirements were not tested separately, and budgets differed. We compare the whole method with a baseline, and AIMS spends more tokens than an agent without it. Part of any difference may come from the extra effort. The experiment that separates these is a 2×2 at equal budget — {written principles, general instruction} $\times$ {checked during design, reviewed afterwards} — counting how often a concept is modelled as a fake instance of another. If reviewing afterwards catches these as often as checking during design, the second requirement in Section 2 is wrong.

Small samples. One run per arm per product, three design products, five small feature additions, one agent platform.

12 Conclusion

A model optimizes for the goal it is given, and coding agents are given correctness. AIMS makes design quality part of the goal: a scored form built from established principles, applied to the design before code is written, both when designing new code and when adding features to existing code.

We could not show that this helps. When adding features, an agent without the method was equally correct, and the judged benefit over a sequence of additions was not confirmed by an automatic measure. For new designs, two rounds of comparison with OpenSpec gave opposite results, because the grades depended on the judge as much as on the design, the judges could tell which method produced each design, and the measure independent of our principles pointed the other way. AIMS also cost several times more.

The argument of Section 2 may still be right; we have not been able to test it. What is missing is a way to measure design quality that gives the same answer twice. Building one is the next step, before any further claim about AIMS.

Availability

Method, experiment packages, run logs and judge reports: <https://github.com/eliezeravihail/aims>.

References

- [1] K. Beck. “For each desired change, make the change easy (warning: this may be hard), then make the easy change.” Twitter/X, 2012.
- [2] M. Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
- [3] D. Coleman, D. Ash, B. Lowther and P. Oman. Using metrics to evaluate software system maintainability. *Computer*, 27(8):44–49, 1994.
- [4] M. Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004.
- [5] M. Fowler. *Refactoring: Improving the Design of Existing Code*. 2nd ed., Addison-Wesley, 2018.
- [6] M. H. Halstead. *Elements of Software Science*. Elsevier, 1977.
- [7] S. Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024.
- [8] J. Huang et al. Large language models cannot self-correct reasoning yet. *ICLR*, 2024; *arXiv:2310.01798*.

- [9] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.
- [10] V. Krakovna et al. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 21 April 2020.
- [11] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. *NeurIPS*, 2023.
- [12] T. J. McCabe. A complexity measure. *IEEE Trans. Software Engineering*, SE-2(4):308–320, 1976.
- [13] L. Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- [14] M. Page-Jones. *What Every Programmer Should Know About Object-Oriented Design*. Dorset House, 1995.
- [15] C. Qian et al. ChatDev: Communicative agents for software development. *ACL*, 2024.
- [16] M. Sharma et al. Towards understanding sycophancy in language models. *arXiv:2310.13548*; *ICLR*, 2024.
- [17] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.
- [18] Q. Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv:2308.08155*, 2023.
- [19] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *NeurIPS*, 2024.
- [20] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [21] L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.
- [22] R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [23] S. McConnell. *Code Complete*. 2nd ed., Microsoft Press, 2004.
- [24] A. Hunt and D. Thomas. *The Pragmatic Programmer*. Addison-Wesley, 1999.
- [25] E. Gamma, R. Helm, R. Johnson and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [26] R. C. Martin. *Clean Architecture*. Prentice Hall, 2017.
- [27] E. Evans. *Domain-Driven Design*. Addison-Wesley, 2003.
- [28] B. Meyer. *Object-Oriented Software Construction*. 2nd ed., Prentice Hall, 1997.
- [29] ISO/IEC 25010:2011. *Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*.
- [30] OWASP Foundation. *OWASP Secure Coding Practices*.
- [31] M. M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980.
- [32] D. L. Parnas. Software aging. *Proc. 16th International Conference on Software Engineering (ICSE)*, 279–287, 1994.
- [33] S. G. Eick, T. L. Graves, A. F. Karr, J. S. Marron and A. Mockus. Does code decay? Assessing the evidence from change management data. *IEEE Trans. Software Engineering*, 27(1):1–12, 2001.
- [34] J. van Gurp and J. Bosch. Design erosion: problems and causes. *Journal of Systems and Software*, 61(2):105–119, 2002.